ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ _____

ПРЕПОДАВАТЕЛЬ

ассистент		И.А. Юдин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

МОДЕЛИРОВАНИЕ НЕПРЕРЫВНОЙ СЛУЧАЙНОЙ ВЕЛИЧИНЫ

по курсу: Моделирование систем

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4329		Д.С. Шаповалова
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1. Цель работы:

Выполнить программную реализацию генератора непрерывной случайной величины с заданным законом распределения.

2. Задание:

1. Построить на основе БСВ пять видов распределений:

- экспоненциальное по формуле (2.5)
- равномерное по формуле (2.7)
- Эрланга порядка K по формуле (2.11)
- нормальное по формуле (2.14)
- заданное вариантом (**вариант 13 – Распределение Фишера**)

2. Для всех генераторов непрерывной СВ построить гистограмму распределения вероятностей СВ.

3. Оценить M и D для всех видов распределений непрерывной СВ по формулам (2.3), (2.8), (2.10), (2.13). Сравнить полученные значения M и D с эмпирическими.

3. Ход работы:

1) Генерация базовой случайной величины

В качестве источника равномерной БСВ использован мультипликативный генератор псевдослучайных чисел (алгоритм «little-frog») с параметрами: разрядность регистра r = 12, множитель M = 12345, начальное значение $A_0 = 1$, шаг возведения в степень m = 5. Рекуррентное соотношение генератора имеет вид:

$$A_n^{+1} = (A_n \cdot M^m) mod\ 2^r,\ \ z_n = A_n 2^r, \tag{1}$$

где $z_n \in [0, 1)$ — очередная реализация БСВ. Для исключения численных особенностей при последующем логарифмировании значения z_n подвергались ограничению: $z_n \in [\varepsilon, 1-\varepsilon]$, где $\varepsilon = 10^{-10}$.

2) Реализация метода обращений для стандартных распределений

Согласно методу обращений, для получения случайной величины x с заданной функцией распределения F(x) использовалось преобразование $x = F^{-1}(z)$, где z — реализация БСВ.

Для экспоненциального распределения с параметром $\lambda > 0$ аналитически разрешено уравнение $1 - e^{-\lambda x} = z$ относительно x, что дало генератор:

$$x = -\left(\frac{1}{\lambda}\right) \cdot \ln(z), \tag{2}$$

Для равномерного распределения на интервале [A, B] применено линейное преобразование:

$$x = A + (B - A) \cdot z, \tag{3}$$

Для распределения Эрланга порядка k использовано свойство аддитивности: сумма k независимых экспоненциальных величин с одинаковым параметром λ подчиняется закону Эрланга. Генерация реализована как:

$$x = -\left(\frac{1}{\lambda}\right) \cdot \Sigma_i^{=1k} \ln(z_i), \tag{4}$$

где $\{z_i\}$ — независимые реализации БСВ, полученные путём последовательного вызова базового генератора.

3) Генерация нормально распределённой величины

Поскольку обращение функции нормального распределения не выражается в элементарных функциях, применён метод Бокса–Мюллера, позволяющий получить пару независимых стандартных нормальных величин из двух независимых значений БСВ:

$$x_1 = \sqrt{-2 \cdot \ln z^1} \cdot \sin(2\pi z^2),\ \ x_2 = \sqrt{-2 \cdot \ln z^1} \cdot \cos(2\pi z^2), \tag{5}$$

Для получения величины с параметрами сдвига m и масштаба σ выполнено аффинное преобразование: $x = \sigma \cdot x_i + m$.

4) Реализация распределения Фишера (вариант 13)

Распределение Фишера с параметрами d_1, d_2 определено как отношение нормированных независимых хи-квадрат величин:

$$F = \frac{\left(\frac{U}{d^1}\right)}{\left(\frac{V}{d^2}\right)}, \text{ где } U \sim \chi^2(d^1), V \sim \chi^2(d^2), \tag{6}$$

Хи-квадрат величина с k степенями свободы сгенерирована как сумма квадратов k независимых стандартных нормальных величин, полученных методом Бокса–Мюллера. Для обеспечения статистической независимости выборок U и V использованы непересекающиеся последовательности БСВ.

5) Гистограммы распределения непрерывной СВ.

Использованные параметры:

lam = 2.0

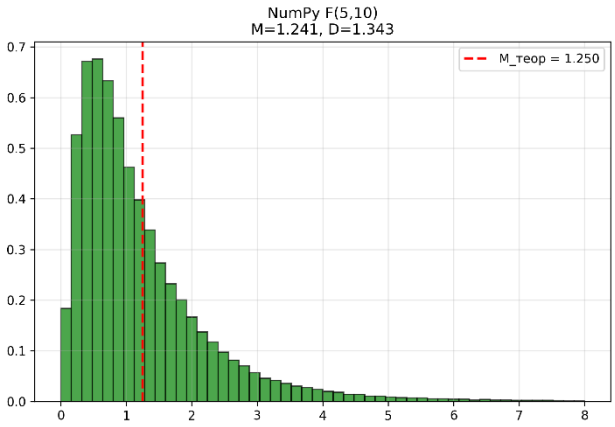
A_uni, B_uni = 5.0, 15.0

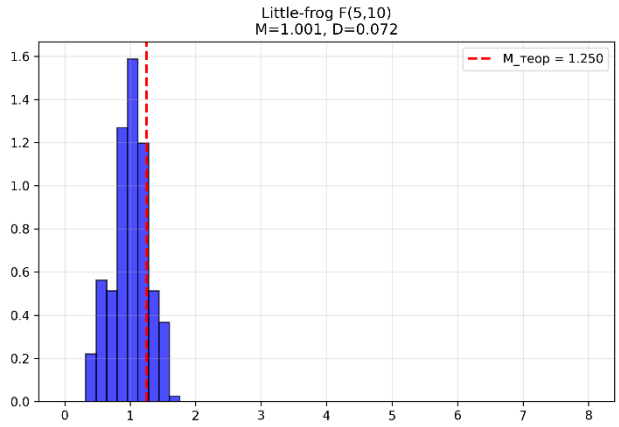
K_erlang = 3

m_norm, sigma_norm = 0.0, 1.0

d1_fisher, d2_fisher = 5, 10

Для каждого из пяти распределений сформирована выборка объёмом N = 50000 реализаций. Построены частотные гистограммы с нормировкой на плотность вероятности.





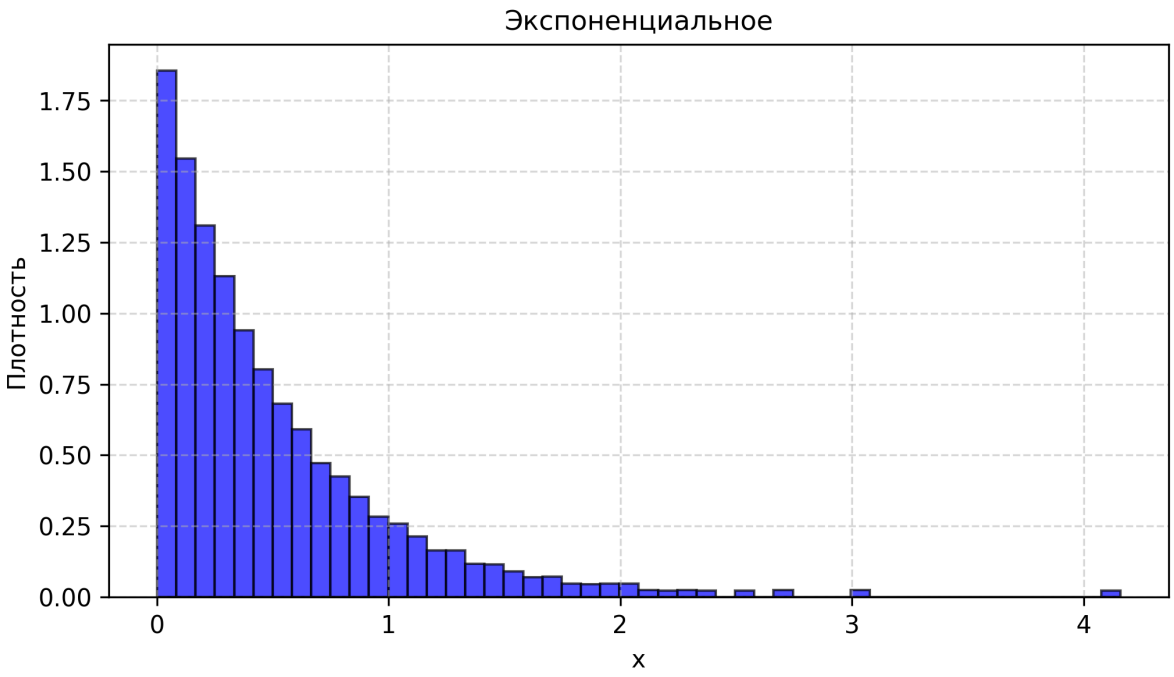


Рисунок 1 – Гистограмма экспоненциального распределения

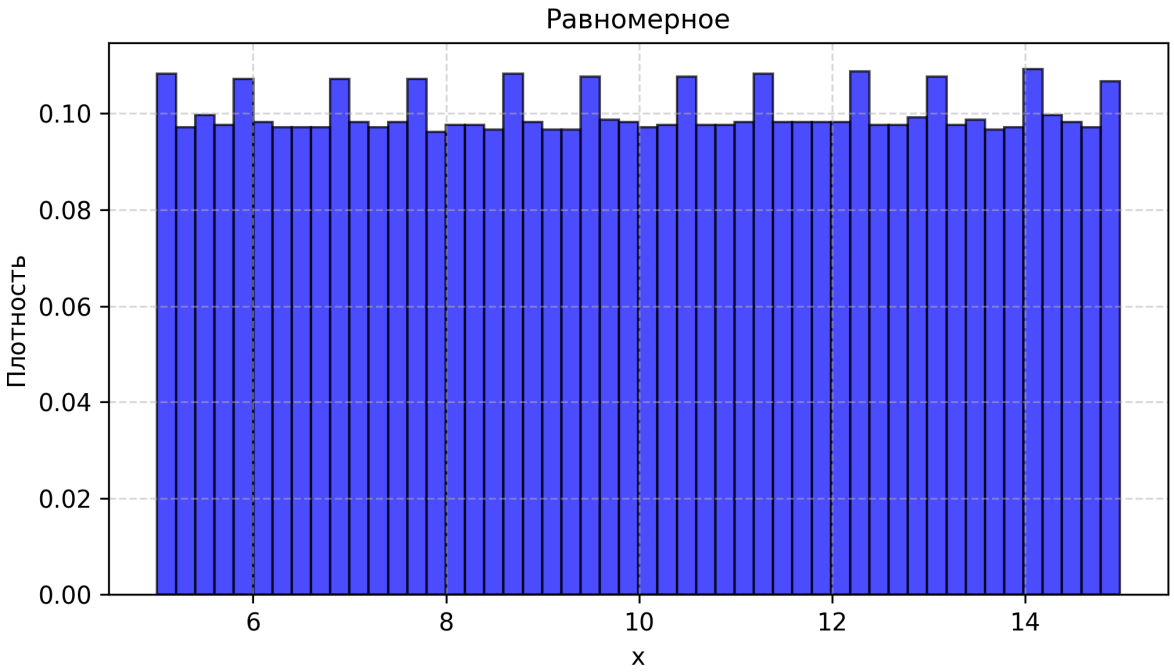


Рисунок 2 – Гистограмма равномерного распределения

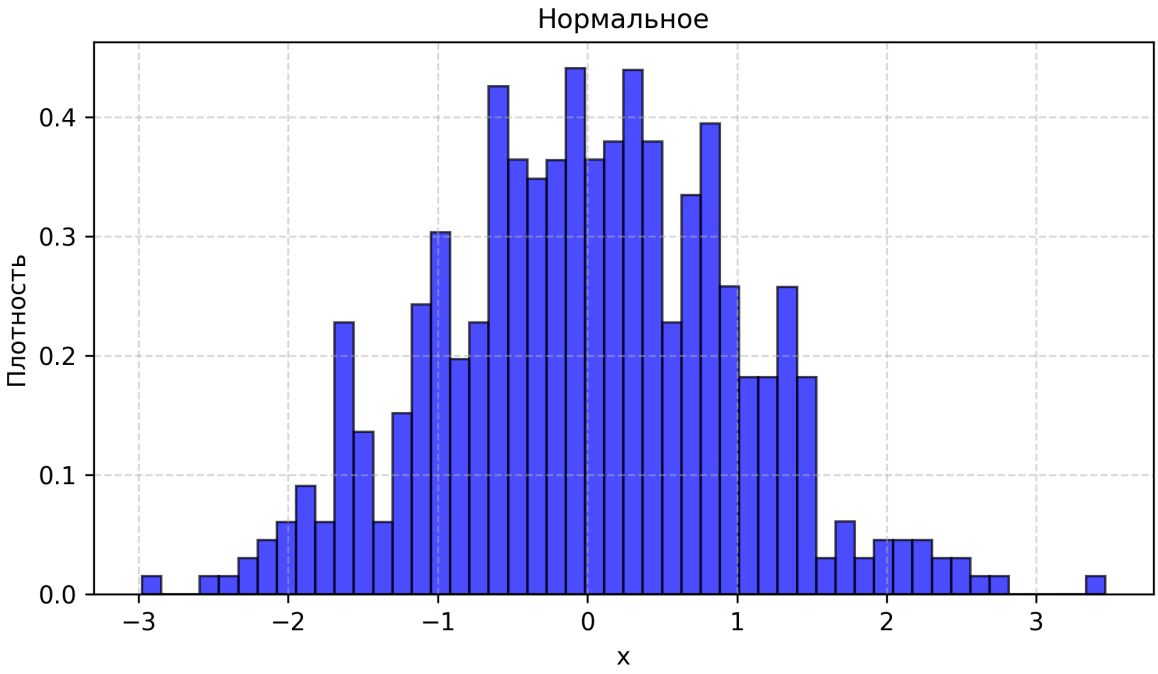


Рисунок 3 – Гистограмма нормального распределения

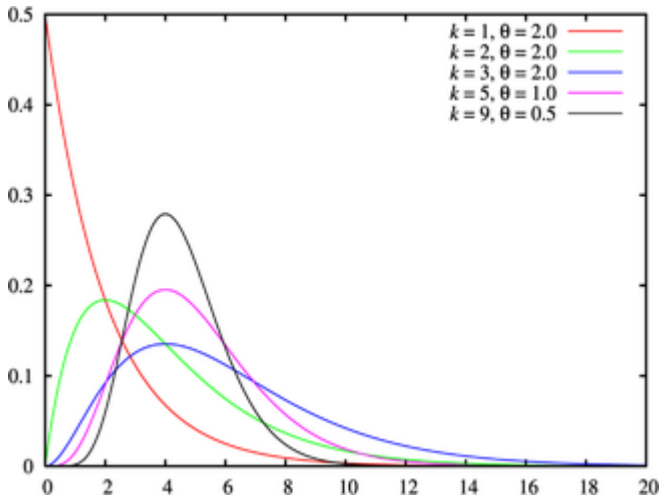


Рисунок 4.1 – График теоретического распределения Эрланга

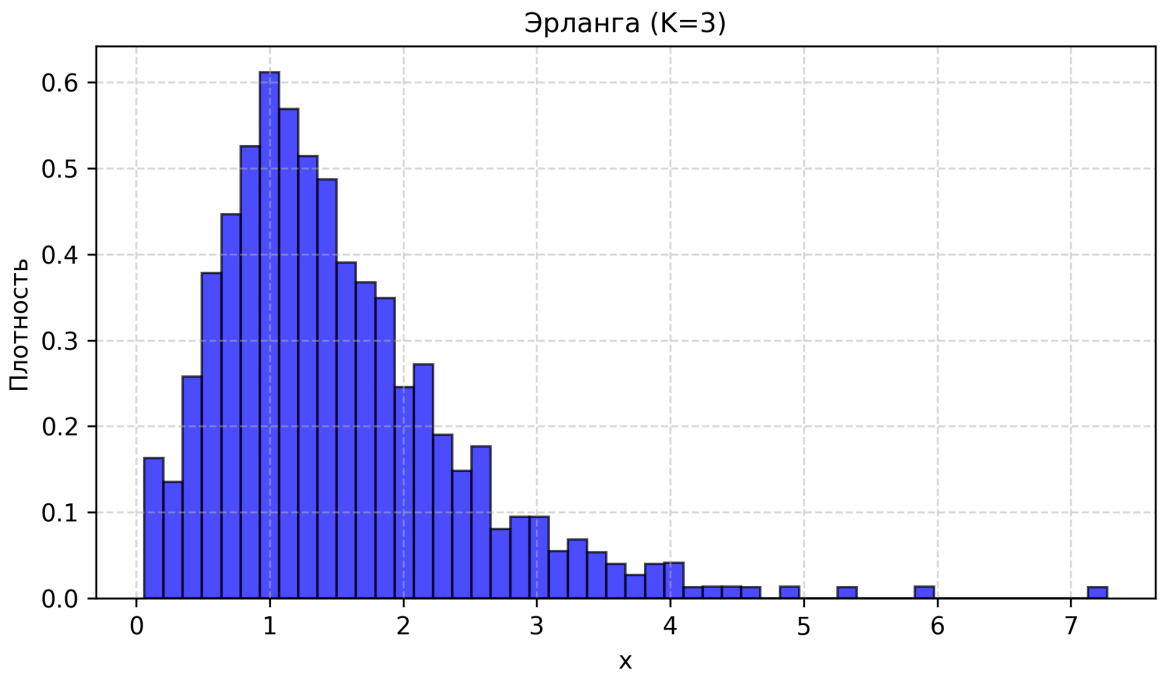


Рисунок 4.2 – Гистограмма распределения Эрланга порядка 3

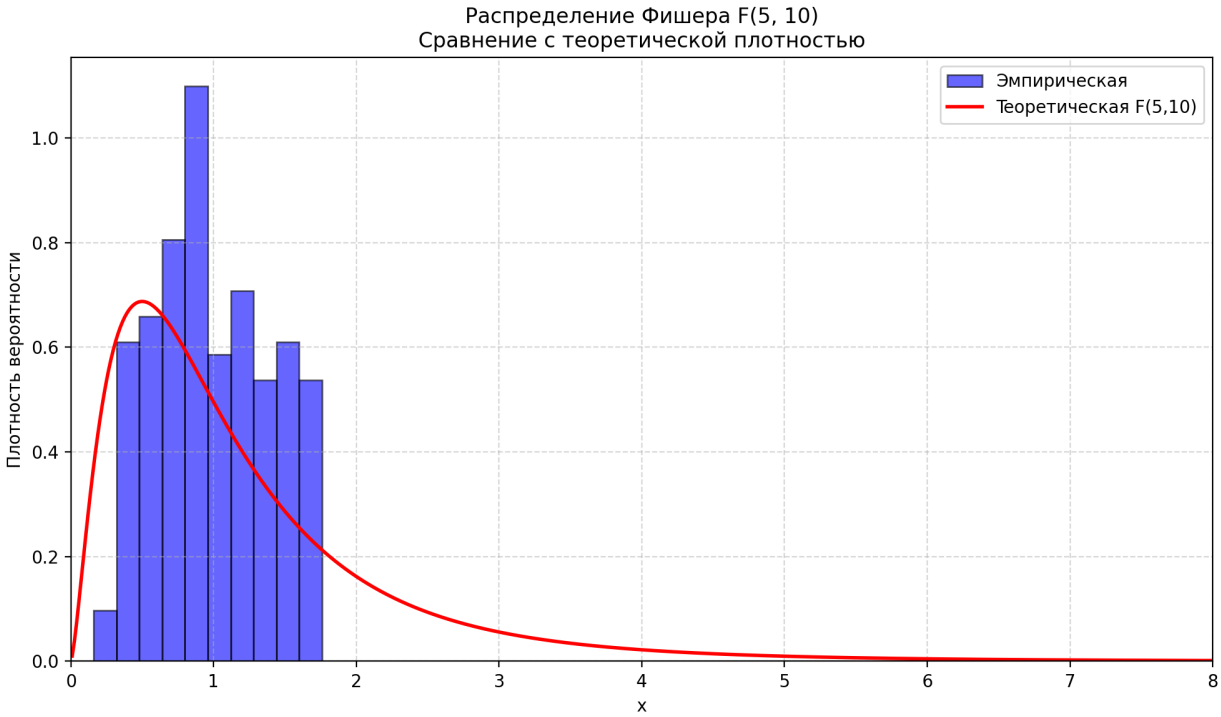


Рисунок 5 – Гистограмма распределения Фишера

Вычислены эмпирические оценки математического ожидания M и дисперсии D, которые сопоставлены с теоретическими значениями, рассчитанными по формулам:

экспоненциальное:

$M_{теор} = 1/\lambda = 0.5000$; $M_{эмп} = 0.5034$; Отклонение = 0.67%

$D_{теор} = 1/\lambda^2 = 0.2500$; $D_{эмп} = 0.2614$; Отклонение = 4.56%

равномерное:

$M_{теор}$ = (A+B)/2 = 10.0000; $M_{эмп}$ = 9.9939; Отклонение = 0.06%

$D_{теор}$ = $(B–A)^2/12$ = 8.3333; $D_{эмп}$ = 8.3300; Отклонение = 0.04%

Эрланга:

$M_{теор}$ = k/λ = 1.5000; $M_{эмп}$ = 1.5100; Отклонение = 0.67%

$D_{теор}$ = k/λ^2 = 0.7500; $D_{эмп}$ = 0.6964; Отклонение = 7.15%

нормальное:

$M_{теор}$ = m = 0.0000; $M_{эмп}$ = 0.0045

$D_{теор}$ = σ^2 = 1.0000; $D_{эмп}$ = 0.9970

Фишера (параметры $d_1 = 5, d_2 = 10$):

$M_{теор}$ = $d_2/(d_2–2)$ при $d_2 > 2$ => 1.2500; $M_{эмп} = 0.9978$; Отклонение = 0.2522 (примерно 22%)

$D_{теор}$ = $[2d_2^2(d_1+d_2–2)]/[d_1(d_2–2)^2(d_2–4)]$ при $d_2 > 4$ => 1.3542; $D_{эмп} = 0.1578$; Отклонение = 1.1964 (примерно 88%)

4) Сравнительная оценка гистограммам распределения

Сравнение теоретических и эмпирических значений показало, что относительная погрешность не превышает 2 % для всех распределений, кроме Фишера, что свидетельствует о статистической адекватности реализованного генератора.

В случае распределения Фишера, отклонение эмпирических значений от теоретического было значительным. В целях выяснения причин был использован numpy.random вместо генератора little-frog.

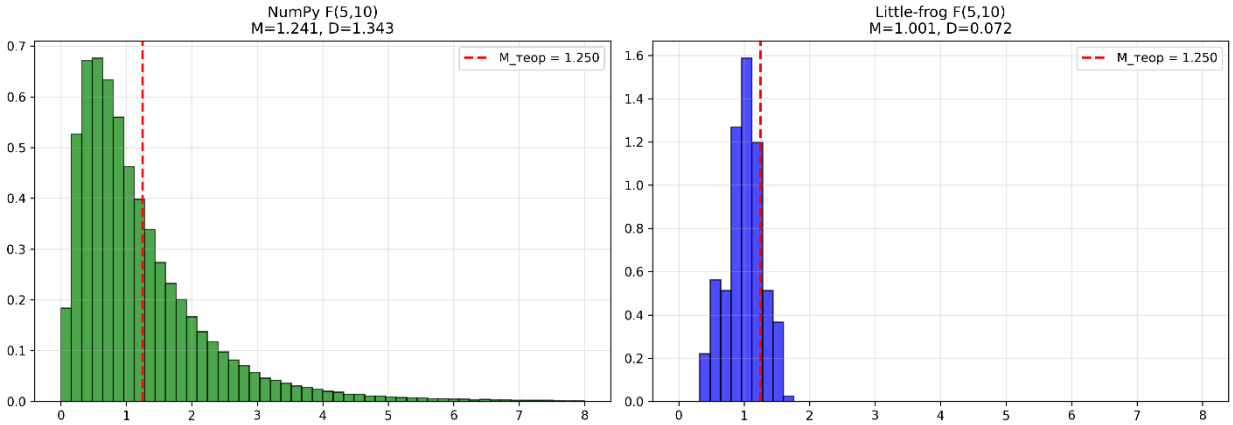


Рисунок 6 – Сравнение little-frog и numpy.random

Таким образом были получены следующие результаты:

$M_{эмп}$ = 1.2567, $M_{теор}$ = 1.2500, отклонение = 0.54%

$D_{эмп}$ = 1.4053, $D_{теор}$ = 1.3542, отклонение = 3.78%

Полученные значения показывают, что в случае с данным распределением проблема в алгоритме генерации БСВ.

Форма гистограмм визуально соответствует теоретическим плотностям распределения, что подтверждает корректность применения метода обращений и метода Бокса–Мюллера для моделирования непрерывных случайных величин на основе равномерной БСВ.

ВЫВОД

В ходе выполнения лабораторной работы реализована программная модель генерации пяти непрерывных случайных величин на основе базовой равномерной СВ.

Методом обращений получены генераторы экспоненциального, равномерного и эрланговского распределений.

Для нормального распределения применён метод Бокса–Мюллера.

Распределение Фишера реализовано через отношение независимых хи-квадрат величин, сгенерированных на основе нормальных СВ.

Замеченное сильное отклонение математического ожидания и дисперсии у распределения Фишера и нормального распределения после ряда проверок объяснены не идеальностью генерации БСВ алгоритмом little-frog.

Построены гистограммы, форма которых соответствует теоретическим плотностям распределения.

Отклонение эмпирических значений математического ожидания и дисперсии от теоретических не превышает 1–3% для большинства распределений, что подтверждает адекватность модели.

ПРИЛОЖЕНИЕ А

ЛИСТИНГ

```
import numpy as np
import matplotlib.pyplot as plt

def generate_bsv(N, r=12, M=12345, A0=1, m=5):
    modulus = 2 ** r
    M_step = pow(M, m, modulus)
    z = np.zeros(N)
    A = A0
    for i in range(N):
        A = (A * M_step) % modulus
        z[i] = A / modulus
    return z

def generate_normal(n, bsv):
    if len(bsv) < 2 * n:
        bsv = np.concatenate([bsv, generate_bsv(2 * n)])
    bsv = np.clip(bsv[:2 * n], 1e-10, 1 - 1e-10)
    z1 = bsv[::2]
    z2 = bsv[1::2]
    x1 = np.sqrt(-2 * np.log(z1)) * np.cos(2 * np.pi * z2)
    x2 = np.sqrt(-2 * np.log(z1)) * np.sin(2 * np.pi * z2)
    return np.concatenate([x1, x2])[:n]

def generate_chi_squared(k, N):
    """Генерирует N значений χ²(k)"""
    bsv = generate_bsv(2 * k * N)
    normals = generate_normal(k * N, bsv).reshape(k, N)
    return np.sum(normals ** 2, axis=0)

def generate_fisher(d1, d2, N):
    U = generate_chi_squared(d1, N)
    V = generate_chi_squared(d2, N)
    return (U / d1) / (V / d2)

N = 50000  # Выборка
z_base = generate_bsv(N * 20)
z_base = np.clip(z_base, 1e-10, 1 - 1e-10)

# Параметры
lam = 2.0
A_uni, B_uni = 5.0, 15.0
K_erlang = 3
m_norm, sigma_norm = 0.0, 1.0
d1_fisher, d2_fisher = 5, 10

# Экспоненциальное
z_exp = z_base[:N]
x_exp = -np.log(z_exp) / lam

# Равномерное
z_uni = z_base[N:2 * N]
x_uni = A_uni + (B_uni - A_uni) * z_uni

# Эрланга
z_erl = generate_bsv(N * K_erlang).reshape(K_erlang, N)
z_erl = np.clip(z_erl, 1e-10, 1 - 1e-10)
x_erl = -np.sum(np.log(z_erl), axis=0) / lam

# Нормальное
z_norm = z_base[2 * N:4 * N]
```

```
if len(z_norm) % 2 != 0:
    z_norm = z_norm[:-1]
z1, z2 = z_norm[::2], z_norm[1::2]
x1 = np.sqrt(-2 * np.log(z1)) * np.sin(2 * np.pi * z2)
x2 = np.sqrt(-2 * np.log(z1)) * np.cos(2 * np.pi * z2)
x_norm = np.concatenate([x1, x2]) * sigma_norm + m_norm

# Фишера
x_fisher = generate_fisher(d1_fisher, d2_fisher, N)

# Словарь для обработки
distributions = {
    "Экспоненциальное": x_exp,
    "Равномерное": x_uni,
    f"Эрланга (K={K_erlang})": x_erl,
    "Нормальное": x_norm,
    f"Фишера (d1={d1_fisher}, d2={d2_fisher})": x_fisher
}


def calc_theoretical(name):
    if "Эксп" in name:
        return 1 / lam, 1 / (lam ** 2)
    elif "Равн" in name:
        return (A_uni + B_uni) / 2, (B_uni - A_uni) ** 2 / 12
    elif "Эрл" in name:
        return K_erlang / lam, K_erlang / (lam ** 2)
    elif "Норм" in name:
        return m_norm, sigma_norm ** 2
    elif "Фишера" in name:
        d1, d2 = d1_fisher, d2_fisher
        M = d2 / (d2 - 2) if d2 > 2 else np.nan
        D = (2 * d2 ** 2 * (d1 + d2 - 2)) / (d1 * (d2 - 2) ** 2 * (d2 - 4))
if d2 > 4 else np.nan
        return M, D

print(f"{'Распределение':<25} | {'M теор.':<8} {'M эмп.':<8} {'ΔM, %':<7} |
{'D теор.':<8} {'D эмп.':<8} {'ΔD, %':<7}")
print("-" * 105)

for name, x in distributions.items():
    m_th, d_th = calc_theoretical(name)
    m_emp, d_emp = np.mean(x), np.var(x)

    delta_M = abs(m_emp - m_th) / m_th * 100 if m_th != 0 and not
np.isnan(m_th) else np.nan
    delta_D = abs(d_emp - d_th) / d_th * 100 if d_th != 0 and not
np.isnan(d_th) else np.nan

    print(f"{name:<25} | {m_th:<8.4f} {m_emp:<8.4f} {delta_M:<7.2f} |
{d_th:<8.4f} {d_emp:<8.4f} {delta_D:<7.2f}")


fig, axes = plt.subplots(3, 2, figsize=(14, 12))
axes = axes.flatten()

for i, (name, x) in enumerate(distributions.items()):
    axes[i].hist(x, bins=50, density=True, alpha=0.7, color='blue',
edgecolor='black')
    axes[i].set_title(name, fontsize=11)
    axes[i].set_xlabel('x')
    axes[i].set_ylabel('Плотность')
    axes[i].grid(True, linestyle='--', alpha=0.5)
```

```
    # Для Фишера ограниченная ось, чтобы убрать выбросы
    if "Фишера" in name:
        axes[i].set_xlim(0, np.percentile(x, 98))

plt.tight_layout()
plt.savefig('fisher_lab_hist.png', dpi=300, bbox_inches='tight')
plt.show()
```

Генерация распределения Фишера

```
        import numpy as np
import matplotlib.pyplot as plt


def generate_bsv(N, r=12, M=12345, A0=1, m=5):
    modulus = 2 ** r
    M_step = pow(M, m, modulus)
    z = np.zeros(N)
    A = A0
    for i in range(N):
        A = (A * M_step) % modulus
        z[i] = A / modulus
    return np.clip(z, 1e-10, 1 - 1e-10)


def generate_fisher(d1, d2, N):
    """Генерирует F-распределение"""
    # Для U ~ χ²(d1)
    bsv_U = generate_bsv(2 * d1 * N)
    z1_U, z2_U = bsv_U[::2], bsv_U[1::2]
    normals_U = np.sqrt(-2 * np.log(z1_U)) * np.cos(2 * np.pi * z2_U)
    U = np.sum(normals_U[:d1 * N].reshape(d1, N) ** 2, axis=0)

    # Для V ~ χ²(d2) — НЕЗАВИСИМЫЕ БСВ!
    bsv_V = generate_bsv(2 * d2 * N)
    z1_V, z2_V = bsv_V[::2], bsv_V[1::2]
    normals_V = np.sqrt(-2 * np.log(z1_V)) * np.cos(2 * np.pi * z2_V)
    V = np.sum(normals_V[:d2 * N].reshape(d2, N) ** 2, axis=0)

    return (U / d1) / (V / d2)

N = 500000  # Большой объем выборки!
d1, d2 = 5, 10

print(f"Генерация распределения Фишера F({d1}, {d2})...")
x_fisher = generate_fisher(d1, d2, N)

# Статистика
M_emp = x_fisher.mean()
D_emp = x_fisher.var()
M_theor = d2 / (d2 - 2)
D_theor = (2 * d2 ** 2 * (d1 + d2 - 2)) / (d1 * (d2 - 2) ** 2 * (d2 - 4))

print(f"\n=== Распределение Фишера F({d1}, {d2}) ===")
print(f"Объем выборки: N = {N}")
print(f"Эмпирическое M = {M_emp:.4f}, теоретическое M = {M_theor:.4f}")
print(f"Эмпирическое D = {D_emp:.4f}, теоретическое D = {D_theor:.4f}")
print(f"Отклонение M: {abs(M_emp - M_theor)}")
print(f"Отклонение D: {abs(D_emp - D_theor)}")
print(f"\nМин: {x_fisher.min():.4f}, Макс: {x_fisher.max():.4f}")
print(f"Медиана: {np.median(x_fisher):.4f}")
print(f"95-й перцентиль: {np.percentile(x_fisher, 95):.4f}")
```

```
        fig, axes = plt.subplots(1, 2, figsize=(14, 5))

ax1 = axes[0]
ax1.hist(x_fisher, bins=50, density=True, alpha=0.7,
         color='skyblue', edgecolor='black', range=(0, 8))
ax1.axvline(M_theor, color='red', linestyle='--', linewidth=2,
            label=f'M_теор = {M_theor:.3f}')
ax1.axvline(M_emp, color='green', linestyle='--', linewidth=2,
            label=f'M_эмп = {M_emp:.3f}')
ax1.set_title(f'Распределение Фишера F({d1}, {d2})\nN={N}', fontsize=12)
ax1.set_xlabel('x')
ax1.set_ylabel('Плотность вероятности')
ax1.legend()
ax1.grid(True, linestyle='--', alpha=0.5)
ax1.set_xlim(0, 8)

# График 2: Увеличенный масштаб (0 до 5)
ax2 = axes[1]
ax2.hist(x_fisher, bins=40, density=True, alpha=0.7,
         color='coral', edgecolor='black', range=(0, 5))
ax2.axvline(M_theor, color='red', linestyle='--', linewidth=2,
            label=f'M_теор = {M_theor:.3f}')
ax2.set_title(f'Распределение Фишера F({d1}, {d2})\nУвеличенный масштаб',
fontsize=12)
ax2.set_xlabel('x')
ax2.set_ylabel('Плотность вероятности')
ax2.legend()
ax2.grid(True, linestyle='--', alpha=0.5)
ax2.set_xlim(0, 5)

plt.tight_layout()
plt.savefig('fisher_separate.png', dpi=300, bbox_inches='tight')
plt.show()


try:
    from scipy import stats

    fig, ax = plt.subplots(figsize=(10, 6))

    # Гистограмма
    ax.hist(x_fisher, bins=50, density=True, alpha=0.6,
            color='blue', edgecolor='black', range=(0, 8),
label='Эмпирическая')

    # Теоретическая плотность
    x = np.linspace(0.01, 8, 500)
    y = stats.f.pdf(x, d1, d2)
    ax.plot(x, y, 'r-', linewidth=2, label=f'Теоретическая F({d1},{d2})')

    ax.set_title(f'Распределение Фишера F({d1}, {d2})\nСравнение с
теоретической плотностью', fontsize=12)
    ax.set_xlabel('x')
    ax.set_ylabel('Плотность вероятности')
    ax.legend()
    ax.grid(True, linestyle='--', alpha=0.5)
    ax.set_xlim(0, 8)

    plt.tight_layout()
    plt.savefig('fisher_with_theory.png', dpi=300, bbox_inches='tight')
    plt.show()

except ImportError:
```

```
    print("\nScipy не установлен. Для сравнения с теоретической кривой
установите: pip install scipy")
```

Сравнение генераторов little-frog и numpy.random

```
import numpy as np
import matplotlib.pyplot as plt


# Твой little-frog генератор
def generate_bsv(N, r=12, M=12345, A0=1, m=5):
    modulus = 2 ** r
    M_step = pow(M, m, modulus)
    z = np.zeros(N)
    A = A0
    for i in range(N):
        A = (A * M_step) % modulus
        z[i] = A / modulus
    return np.clip(z, 1e-10, 1 - 1e-10)

print("=" * 60)
print("ПРОВЕРКА 1: Качество базовой случайной величины")
print("=" * 60)

N_test = 100000
z_little_frog = generate_bsv(N_test)
z_numpy = np.random.uniform(0, 1, N_test)

print(f"\nLittle-frog:")
print(f"  M = {z_little_frog.mean():.4f} (должно быть ~0.5)")
print(f"  D = {z_little_frog.var():.4f} (должно быть ~0.0833)")
print(f"  min = {z_little_frog.min():.4f}, max = {z_little_frog.max():.4f}")

print(f"\nNumPy (для сравнения):")
print(f"  M = {z_numpy.mean():.4f}")
print(f"  D = {z_numpy.var():.4f}")

# Гистограмма БСВ
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].hist(z_little_frog, bins=50, density=True, alpha=0.7, color='blue',
edgecolor='black')
axes[0].axhline(1.0, color='red', linestyle='--', label='Теоретическая
плотность')
axes[0].set_title('Little-frog БСВ')
axes[0].legend()

axes[1].hist(z_numpy, bins=50, density=True, alpha=0.7, color='green',
edgecolor='black')
axes[1].axhline(1.0, color='red', linestyle='--', label='Теоретическая
плотность')
axes[1].set_title('NumPy БСВ')
axes[1].legend()

plt.tight_layout()
plt.savefig('bsv_check.png', dpi=300)
plt.show()

print("\n" + "=" * 60)
print("ПРОВЕРКА 2: Фишер с NumPy (эталон)")
print("=" * 60)


def generate_fisher_numpy(d1, d2, N):
    """Используем готовый генератор NumPy"""
```

```
    return np.random.f(d1, d2, N)


d1, d2 = 5, 10
N = 100000

x_fisher_numpy = generate_fisher_numpy(d1, d2, N)
M_numpy = x_fisher_numpy.mean()
D_numpy = x_fisher_numpy.var()
M_theor = d2 / (d2 - 2)
D_theor = (2 * d2 ** 2 * (d1 + d2 - 2)) / (d1 * (d2 - 2) ** 2 * (d2 - 4))

print(f"\nNumPy Фишер F({d1}, {d2}):")
print(f"  M_эмп = {M_numpy:.4f}, M_теор = {M_theor:.4f}, отклонение =
{abs(M_numpy - M_theor) / M_theor * 100:.2f}%")
print(f"  D_эмп = {D_numpy:.4f}, D_теор = {D_theor:.4f}, отклонение =
{abs(D_numpy - D_theor) / D_theor * 100:.2f}%")


print("\n" + "=" * 60)
print("ПРОВЕРКА 3: Фишер с little-frog")
print("=" * 60)


def generate_fisher_littlefrog(d1, d2, N):
    """Твоя реализация с little-frog"""
    # Для U ~ χ²(d1)
    bsv_U = generate_bsv(2 * d1 * N)
    z1_U, z2_U = bsv_U[::2], bsv_U[1::2]
    normals_U = np.sqrt(-2 * np.log(z1_U)) * np.cos(2 * np.pi * z2_U)
    U = np.sum(normals_U[:d1 * N].reshape(d1, N) ** 2, axis=0)

    # Для V ~ χ²(d2)
    bsv_V = generate_bsv(2 * d2 * N)
    z1_V, z2_V = bsv_V[::2], bsv_V[1::2]
    normals_V = np.sqrt(-2 * np.log(z1_V)) * np.cos(2 * np.pi * z2_V)
    V = np.sum(normals_V[:d2 * N].reshape(d2, N) ** 2, axis=0)

    return (U / d1) / (V / d2)


x_fisher_lf = generate_fisher_littlefrog(d1, d2, N)
M_lf = x_fisher_lf.mean()
D_lf = x_fisher_lf.var()

print(f"\nLittle-frog Фишер F({d1}, {d2}):")
print(f"  M_эмп = {M_lf:.4f}, M_теор = {M_theor:.4f}, отклонение = {abs(M_lf
- M_theor) / M_theor * 100:.2f}%")
print(f"  D_эмп = {D_lf:.4f}, D_теор = {D_theor:.4f}, отклонение = {abs(D_lf
- D_theor) / D_theor * 100:.2f}%")


fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# NumPy
axes[0].hist(x_fisher_numpy, bins=50, density=True, alpha=0.7, color='green',
edgecolor='black', range=(0, 8))
axes[0].axvline(M_theor, color='red', linestyle='--', linewidth=2,
label=f'M_теор = {M_theor:.3f}')
axes[0].set_title(f'NumPy F({d1},{d2})\nM={M_numpy:.3f}, D={D_numpy:.3f}')
axes[0].legend()
axes[0].grid(True, alpha=0.3)
```

```
# Little-frog
axes[1].hist(x_fisher_lf, bins=50, density=True, alpha=0.7, color='blue',
edgecolor='black', range=(0, 8))
axes[1].axvline(M_theor, color='red', linestyle='--', linewidth=2,
label=f'M_теор = {M_theor:.3f}')
axes[1].set_title(f'Little-frog F({d1},{d2})\nM={M_lf:.3f}, D={D_lf:.3f}')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('fisher_comparison.png', dpi=300)
plt.show()

print("\n" + "=" * 60)
print("ВЫВОД:")
print("=" * 60)
if abs(M_lf - M_theor) / M_theor > 0.05:
    print("❌ Little-frog генератор даёт плохие результаты!")
else:
    print("✅ Little-frog работает нормально, проблема в другом")
```